

LAPORAN PRAKTIKUM
PEMROGRAMAN BERBASIS OBJEK
Keamanan & Integritas Data (Enkapsulasi)

Disusun Oleh:

Nofri Ilham

2511531013

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

Ezza Andini



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

A. Tahapan Pelaksanaan Praktikum

Langkah 1: Mengunci Entitas Transaksi (Transaksi.java)

Class Transaksi mewakili satu catatan transaksi (setor atau tarik). Data transaksi tidak boleh diubah setelah dibuat (*immutable*), sehingga tahapannya sebagai berikut:

1. Semua atribut (idTransaksi, jenis, nominal) diubah menjadi private. Dengan begitu atribut tidak bisa diakses langsung dari luar class, misalnya lewat `trx.nominal = 0`.
2. Constructor Transaksi(String id, String Jenis, double nominal) dibuat public sebagai satu-satunya jalur untuk mengisi nilai atribut saat objek dibuat.
3. Hanya Getter yang dibuat: `getIdTransaksi()`, `getJenis()`, dan `getNominal()`. Data hanya bisa dibaca (read-only). Setter tidak dibuat agar data transaksi tidak bisa dimanipulasi setelah tercatat.
4. Method `cetakDetail()` dibuat public untuk menampilkan detail transaksi.



```
package Pertemuan3;

public class Transaksi {
    //1.Mengubah semua atribut menjadi private
    private String idTransaksi;
    private String jenis;
    private double nominal;

    //Konstruktor
    public Transaksi(String id,String Jenis, double nominal )
    {
        this.idTransaksi = id;
        this.jenis=Jenis;
        this.nominal =nominal;
    }

    //2. Hanya menyediakan Getter (Read-Only)
    public String getIdTransaksi() {return idTransaksi;}
    public String getJenis() {return jenis;}
    public double getNominal() { return nominal;}
    public void cetakDetail() {
        System.out.println("ID: "+ idTransaksi +" | Jenis: "+ jenis + " | Nominal:
Rp"+ nominal);
    }
}
```

Langkah 2: Mengunci Entitas Rekening dan Menambahkan PIN (Rekening.java)

1. Atribut dikunci dengan private: nomorRekening, namaPemilik, saldo, dan riwayatTransaksi. Ditambahkan atribut baru pin (private String pin) sebagai data sensitif.
2. Constructor dimodifikasi agar menerima parameter pinAwal. Di dalamnya ada validasi: jika panjang PIN 6 karakter maka PIN disimpan, jika tidak maka muncul peringatan dan dipakai PIN default 123456.
3. ArrayList riwayatTransaksi diinisialisasi di dalam constructor agar tidak terjadi NullPointerException.
4. Getter hanya dibuat untuk atribut yang boleh dibaca publik, yaitu getNomorRekening() dan getNamaPemilik().
5. Method otentikasi(String inputPin) dibuat public dengan nilai kembalian boolean. Method ini membandingkan PIN input dengan PIN asli di dalam class, sehingga PIN tidak perlu dibuka ke luar.
6. Method setorTunai(), tariktunai(), cekInformasi(), dan cetakMutasi() tetap dipertahankan dan bersifat public karena merupakan layanan yang dipanggil dari luar class.

```

package Pertemuan3;
import java.util.ArrayList;

public class Rekening {
    //1. Mengunci atribut dengan 'private'
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin; // Data Sensitif

    //Implementasi Asosiasi (1-to-many)
    private ArrayList<Transaksi> riwayatTransaksi;

    //2. Modifikasi Constructor untuk menerima PIN awal
    public Rekening (String nomor, String nama, double saldoAwal, String pinAwal){
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        //Validasi PIN di dalam Constructor
        if(pinAwal.length()==6) {
            this.pin=pinAwal;
        }else {
            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN
default 123456");
            this.pin="123456";
        }

        //Wajib meninisialisasi ArrayList di dalam konstruktor agar tidak
        NullPointerException
        this.riwayatTransaksi=new ArrayList<>();
        System.out.println(" Rekening atas nama " + namaPemilik + " Berhasil dibuat
");
    }

    //3. Getter untuk atribut yang diizinkan dibaca publik
    public String getNomorRekening() { return nomorRekening;}
    public String getNamaPemilik() { return namaPemilik;}

    //4. Method Otentikasi Internal (Validasi enkapsulasi)
    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }
}

```

Mengapa kita tidak membuat method setSaldo() dan getPin() ?

Method setSaldo() sengaja tidak dibuat karena akan membuka celah bagi siapa saja untuk mengubah saldo rekening secara langsung, misalnya dengan memanggil setSaldo(9999999), tanpa melalui proses validasi dan tanpa meninggalkan catatan pada riwayat transaksi.

Method getPin() juga tidak dibuat karena PIN merupakan data yang sangat sensitif. Apabila PIN dapat dibaca dari luar class, maka fungsinya sebagai pengaman akan hilang, sebab siapa pun dapat mengetahui dan menyalahgunakannya.

B. Latihan/Tugas

1. Menu 1 (Buka Rekening): Sebelum menginstansiasi objek baru, minta user menginputkan PIN (String angka 6 digit).

Pada program, setelah nomor rekening dan nama pemilik diinput, user diminta memasukkan PIN. PIN dibaca dengan tipe String, bukan int, agar angka nol di depan (misalnya 012345) tidak hilang.

```
System.out.println("Masukkan PIN(Harus 6 Digit)");
String pin = input.nextLine();
```

Selanjutnya PIN divalidasi menggunakan *regular expression* `\\d{6}`, yang berarti PIN harus terdiri dari tepat 6 karakter dan seluruhnya angka.

Selama format belum sesuai, program menampilkan pesan kesalahan dan meminta PIN diinput ulang dengan perulangan `while`.

```
while (!pin.matches("\\d{6}")) {
    System.out.println("Pin harus terdiri dari 6 angka! ");
    System.out.println("Masukkan PIN(Harus 6 Digit)");
    pin = input.nextLine();
}
```

Setelah PIN valid dan saldo awal diinput, objek Rekening dibuat dengan constructor 4 parameter, yaitu nomor, nama, saldo, dan PIN. Objek tersebut disimpan ke `daftarRekening` dan dijadikan `akunAktif`.

```
Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);
daftarRekening.add(rekeningBaru);
akunAktif = rekeningBaru;
```

2. Keamanan Transaksi: Perbarui Menu 3 (Tarik Tunai) dan Menu 6 (Cetak Mutasi). Sebelum sistem mengeksekusi kedua method tersebut, sistem harus meminta user memasukkan PIN.

- Panggil fungsi `akunAktif.otentikasi(pinYangDiinput)`.
- Jika mengembalikan nilai `true`, jalankan transaksi.
- Jika `false`, batalkan dan cetak: "Akses Ditolak: PIN yang Anda masukkan salah!".

Menu 3 (Tarik Tunai). Pada program, terlebih dahulu dilakukan pengecekan apakah `akunAktif` masih `null`. Jika ya, ditampilkan pesan error agar tidak terjadi `NullPointerException`.



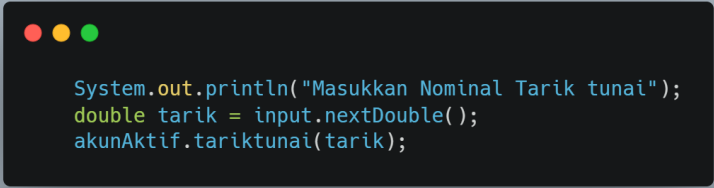
```
if (akunAktif == null) {  
    System.out.println("Error: Anda belum membuka rekening!");  
}
```

Jika rekening sudah ada, user diminta memasukkan PIN, lalu PIN tersebut diverifikasi dengan memanggil method `otentikasi()` milik `akunAktif`.



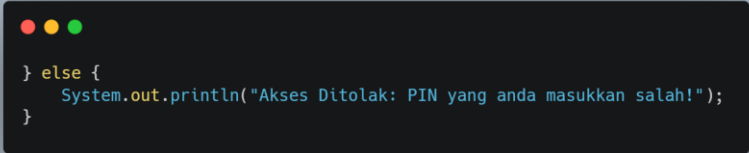
```
System.out.println("Masukkan PIN");  
String pinTarik = input.nextLine();  
if (akunAktif.otentikasi(pinTarik))
```

Apabila `otentikasi()` mengembalikan `true`, program meminta nominal penarikan lalu memanggil method `tarikTunai()`. Pengecekan nominal minimal dan kecukupan saldo tetap dilakukan di dalam class `Rekening`.



```
System.out.println("Masukkan Nominal Tarik tunai");
double tarik = input.nextDouble();
akunAktif.tariktunai(tarik);
```

Apabila `otentikasi()` mengembalikan `false`, transaksi dibatalkan dan method `tariktunai()` tidak dipanggil sama sekali.



```
} else {
    System.out.println("Akses Ditolak: PIN yang anda masukkan salah!");
}
```

Menu 6 (Cetak Mutasi). Alur pada menu ini sama dengan Menu 3. Program mengecek `akunAktif`, meminta PIN, lalu memanggil `otentikasi()`. Jika PIN benar, method `cetakMutasi()` dijalankan untuk menampilkan riwayat transaksi. Jika salah, ditampilkan pesan penolakan.



```
System.out.println("Masukkan PIN");
String pinTarik = input.nextLine();
if (akunAktif.otentikasi(pinTarik)) {
    akunAktif.cetakMutasi();
} else {
    System.out.println("Akses Ditolak: PIN yang anda masukkan salah!");
}
```

C. Link Repository

https://github.com/nofriilham/PrakPBO_C_2511531013/tree/Pertemuan3